جامعة الأمير سلطان
PRINCE SULTAN
UNIVERSITY

IS311 Web Development

Week 06 - Lecture 07

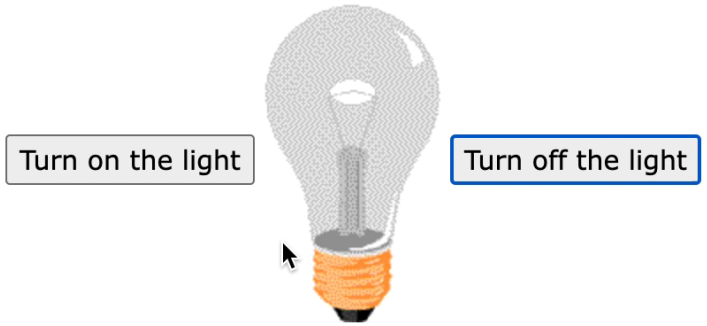
Dr. Abbas Malik
amaalik@psu.edu.sa

JavaScript

- Define behviours for an HTML page
- Allow interactivity from user with elements

JavaScript - change attributes

```
<!DOCTYPE html>
<html>
<body>

<h2>What Can JavaScript Do?</h2>

<p>JavaScript can change HTML attribute values.</p>

<p>In this case JavaScript changes the value of the src (source) attribute of an image.
</p>

<button onclick="document.getElementById('myImage').src='pic_bulbon.gif'">Turn on the
light</button>

<img id="myImage" src="pic_bulboff.gif" style="width:100px">

<button onclick="document.getElementById('myImage').src='pic_bulboff.gif'">Turn off the
light</button>

</body>
</html>
```



JavaScript

- Change HTML Contents

```
document.getElementById('demo').innerHTML = 'Hello JavaScript';
```

- Change HTML Style

```
document.getElementById("demo").style.fontSize = "35px";
```

- Hide or show HTML Elements

More on Display Property

```
document.getElementById("demo").style.display = "none";
```

```
document.getElementById("demo").style.display = "block";
```

Where to JavaScript

- <script> … </script> tag
- Always writen in <script> tag with a closing tag
- In
  - Head
  - Body
  - External File
- Execute sequentially

JavaScript in Head or body

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script type="text/JavaScript">
        console.log("Logging info on console from JavaScript");
        console.log(x);
    </script>
    <title>JavaScript</title>
</head>
<body>
    <h1>JavaScript Introduction!</h1>
    <script type="text/JavaScript">
        console.log("Logging info in body");
        console.log(x);
    </script>
</body>
</html>
```

JavaScript in External File

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="JavaScript/first.js"></script>
    <script type="text/JavaScript">
        console.log("Logging info on console from JavaScript");
        console.log(x);
    </script>
    <title>JavaScript</title>
</head>
```

External JavaScript

- Separates HTML and Code
- Easier to read and maintain
- Cached JavaScritp files can speed up page loads
- Can be used external JS file from the internet like BOOTSTRAP

Programming

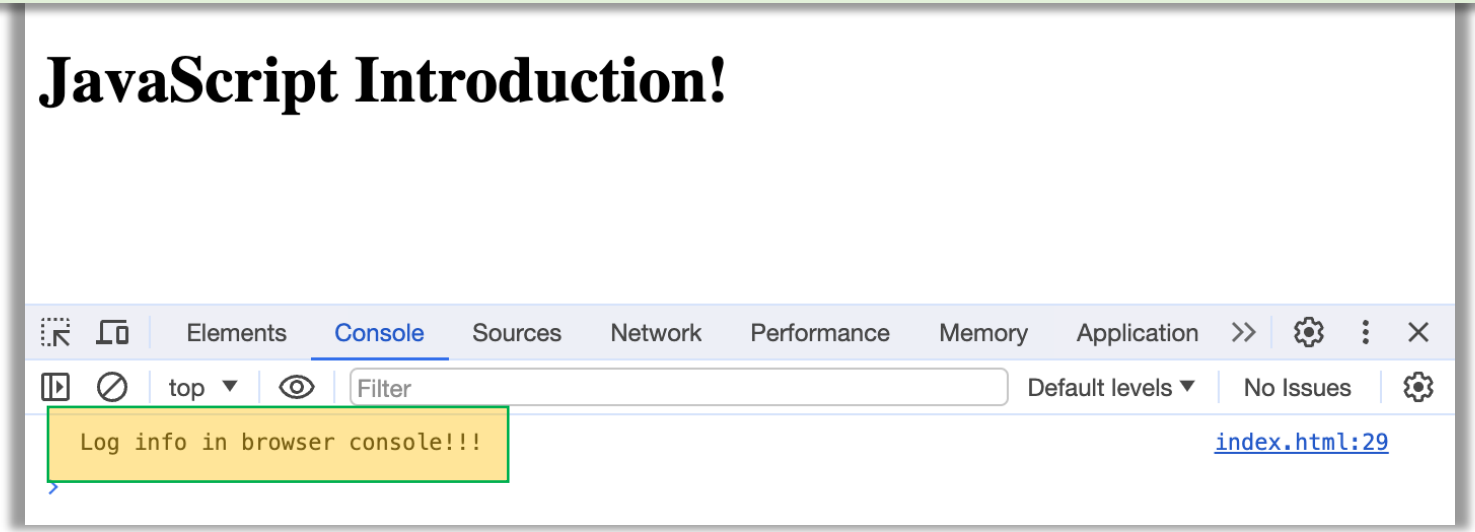
- Varibles
- Loop: while, for, do while
- Conditional statements: if, case
- Methods/Functions
- More…

JavaScript Output

- console.log() – write info in browser console
- window.alet() – write info in alert box
- document.write() – write into the HTML
- innerHTML – write into an element

Console Output

```
<body>
<h1>JavaScript Introduction!</h1>
<script>
  console.log("Log info in browser console!!!");
</script>
</body>
```



Alert Output

```
alert("alert method");
```

```
<script>
window.alert("it is a JS Alert");
</script>
```

HTML Output

```
<body>
  <h1>JavaScript Introduction!</h1>
  <script>
    window.alert("it is a JS Alert");
    document.write("Writing in HTML from JS");
  </script>
</body>
```

Element Output

```
<body>
<h1 id="demo"></h1>
<script>
document.getElementById("demo").innerHTML = "Heading from JS";
</script>
</body>
```

Heading from JS

Statement

- An executable instruction
- ; - separates statements from eachother
- Literals
  - Numbers
  - Strings in single or double quotes
- Variables – used to store data

Syntax

- Declare variables `var, let, const`
- Operators `+ - * / =`
- Experssion `5 * 10, x + y`
  - `“Abbas” + ‘ ’ + ‘Malik’`
- Keywords – words reserved for the language
- Comments `// line comment`
  - `/** */`

Syntax

- Identifiers – name of variables, function, object, etc.
  - Must begins with a letter (A-Z, a-z), or dollar sign ($) or an underscore (_)
- Case senstive language
- Hyphens: first-name
- Underscore: first_name
- Upper Camel Case: FirstName
- Lowe Camel Case: firstName

Variables `var let const`

- 4 ways to declare variables
- Automatic
- Using var
- Using let
- Using const

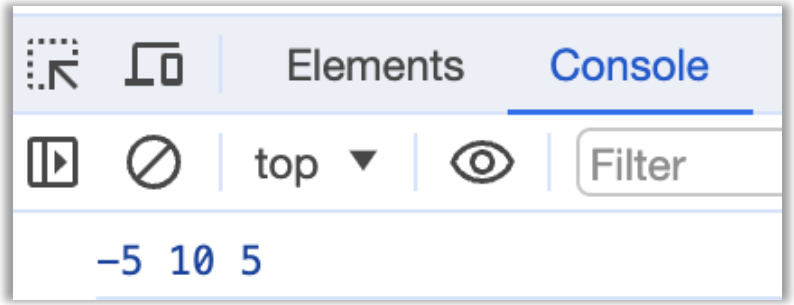
Dynamic Types

Variable

Automatic Declaration

```
y = 10;
console.log(y);
```

```
x = -5;
y = 10;
z = x + y;
console.log(x, y, z);
```



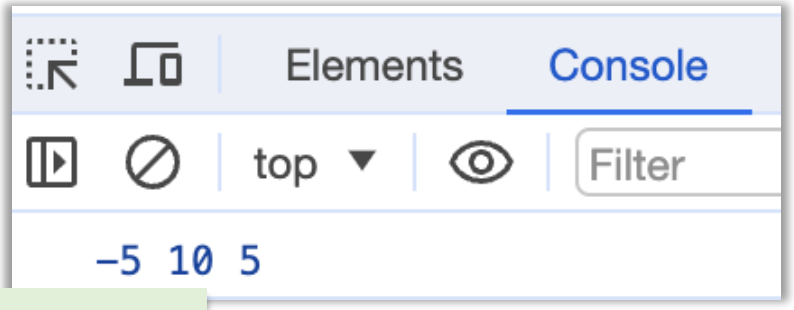
Variable

```
var y = 10;
console.log(y);
```

Using var

- Used from 1995 to 2015
- Should be used for code written for older browsers
- Declare varaibles with **global scope**
- **Equivalent to Automatic**

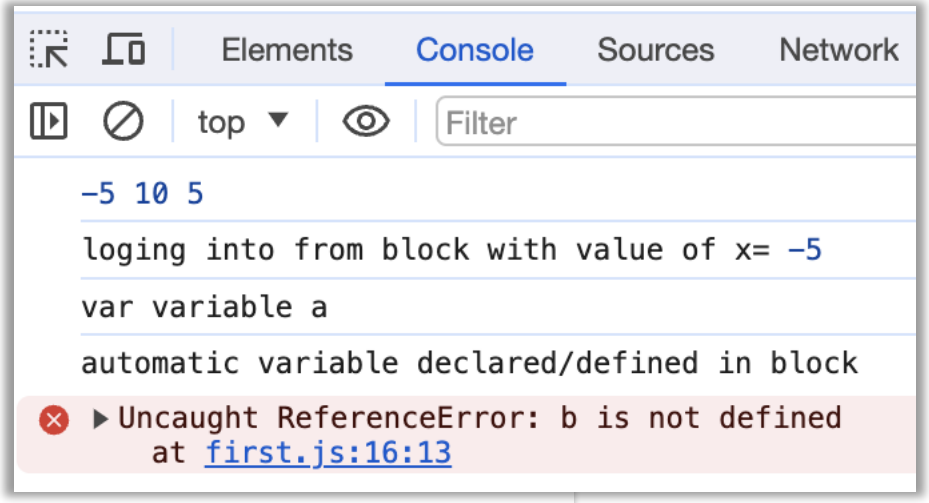
```
var x = -5;
var y = 10;
var z = x + y;
console.log(x, y, z);
```




```
x = -5;
y = 10;
z = x + y;
```

Variable Scope

```
x = -5; // global scope
y = 10; // global scope
z = x + y; // global scope
console.log(x, y, z);

{   // block start
    console.log('loging into from block with value of x=', x);
    var a = "var variable a"; // global scope
    let b = "let variable b in block"; // block scope
    c = "automatic variable declared/defined in block"; // global scope
} // block ends
console.log(a);
console.log(c);
console.log(b);
```



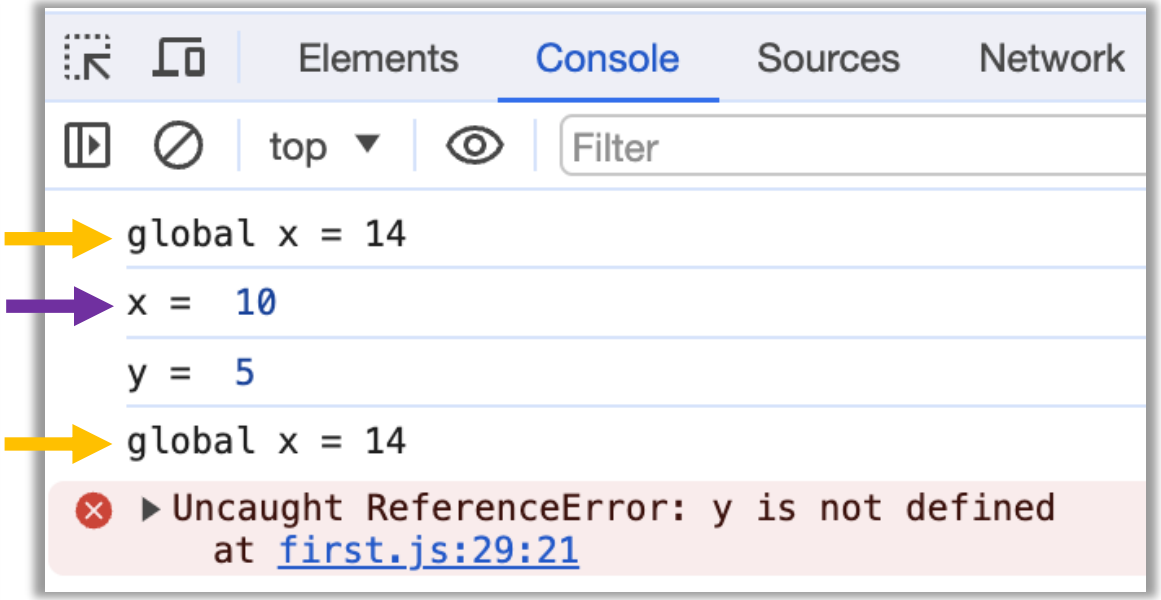
Variable Scope

```
x = 14; // global scope
{
        let y = x + 2;
        {
                let z = x + y;
                {
                        let a = 8;
                        console.log(x);
                        console.log(y);
                        console.log(z);
                        console.log(a);
                }
        }
}
```

Variable

Using let

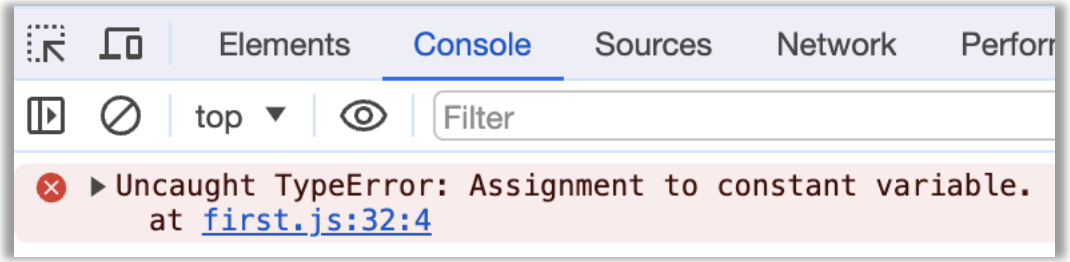
```
x = 14;
console.log('global x = ' + x);
{
        let x = 10;
        console.log('x = ', x);
        let y = 5;
        console.log('y = ', y);
}
console.log('global x = ' + x);
console.log('y = ', y);
```



More details

Variable - Using const

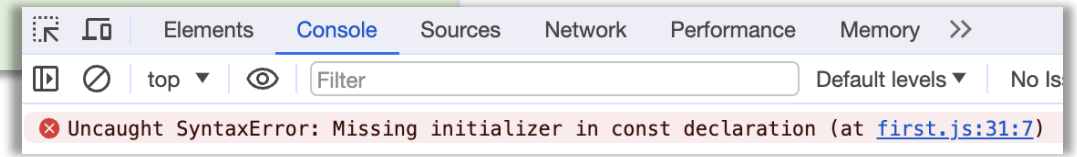
- Cannot reassign value




```
const PI = 3.141592653589793;
PI = 3.14; // This will give an error
```

- Block scope
- Must be assign value at declaration

```
const PI; // This will give an error
PI = 3.14;
```



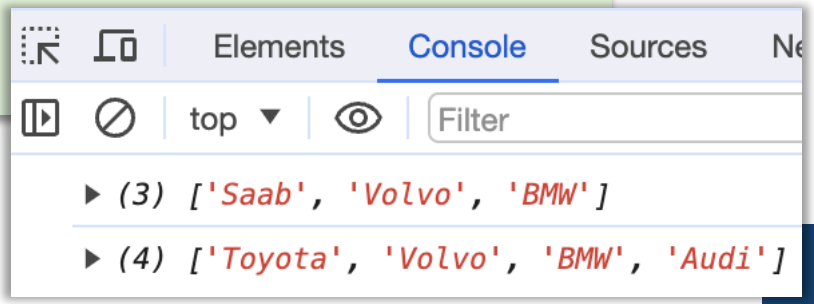
Variable – Using const

- Always declare a variable with **const** when the value should not be changed.
  - A new **Array**
  - A new **Object**
  - A new **Function**
  - A new **RegExp**

```
cars = [1, 2, 3];
```

```
Uncaught TypeError: Assignment to constant variable.
    at first.js:35:6
```

```
// You can create a constant array:
const cars = ["Saab", "Volvo", "BMW"];
console.log(cars);
// You can change an element:
cars[0] = "Toyota";
// You can add an element:
cars.push("Audi");
console.log(cars);
```



const Object

```
// You can create a const object:
const car = {type:"Fiat", model:"500", color:"white"};
console.log(car);
// You can change a property:
car.color = "red";
// You can add a property:
car.owner = "Johnson";
console.log(car);
```


```
car = {type:"Volvo", model:"EX60", color:"red"};
```

More details

Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
**	Exponentiation (ES2016)
/	Division
%	Modulus (Division Remainder)
++	Increment
--	Decrement

Assignment Operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Comparison Operators

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Used for

- Consitional statements like **IF**
- Conditions for Loops

Logical Operators

Operator	Description
&&	logical and
||	logical or
!	logical not

More details

8 Data Types

- String – single or double quotes
- Number – 87, 8.97
- Bigint – ES2020
- Boolean
- Undefined
- Null
- Symbol
- Object

```
let y = 123e5;    // 12300000
let z = 123e-5;   // 0.00123
```

```
let x = 1234567890123456789012345n;
let y = BigInt(1234567890123456789012345)
```

```
let x = true;
let y = false;
```

```
let car;
```

```
let abc = undefined;
if(abc === undefined){
        console.log("undefined");
}
```

An **Object** can be **Null**

Symbol Data Type

- Symbols are unique and immutable
- Used as unique identifiers in objects and classes
- Used to create private properties and methods in classes
- Useful for creating constants for sharing across different parts of your code

Symbol Data Type

```
const mySymbol = Symbol();
const myObject = {
        [mySymbol]: 'Hello World'
};
console.log(myObject[mySymbol]);
```

More details

Data Types

- Objects can be defined using { }
- typeof Operator to find type of a variable

```
num = 78.98;
console.log(typeof num);
str = "a string";
console.log(typeof str);
```

Functions

- A **named block** of code designed to perform a particular task
- Executed when invoked in the code
- Syntax

Keyword — Function name — Optional 0 or more parameter list

```
function function_name (parameter1, parameter2){
        // code to perform a specific task
        // body of function or function block
}
```

Function

- Invoking or Executing function

```
function function_name (parameter1, parameter2){
        // code to perform a specific task
        // body of function or function block
}
```

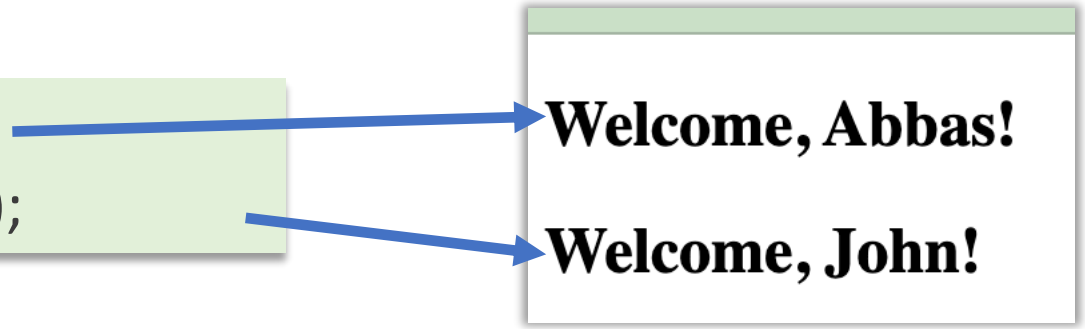
```
function_name();
function_name(23);
function_name(23, 45);
```

Function parameters are optional

Function

```
function greeting (name){
        if(name === undefined){
                document.write("<h2>Welcome, Abbas!</h2>");
        } else {
                document.write("<h2>Welcome, " + name + "!</h2>");
        }
}
```

```
greeting();
greeting("John");
```



Function - return

- On a return statement, the function stops executing

```
function greeting (name){
    if(name === undefined){
        document.write("<h2>Welcome, Abbas!</h2>");
        return;
    }
    document.write("<h2>Welcome, " + name + "!</h2>");
}
```

Function - return

- A function can return a value

```
function add(num1, num2){
    if(num1 === undefined || num2 === undefined){
        return "Please provide two numbers to add them!!!";
    }
    return num1 + num2;
}
```

```
x = 45;
y = 67;
document.write("<h3> sum of " + x + " and " + y + " is " + add(x, y) + ”</h3>");
```

sum of 45 and 67 is 112

Function - return

- A function can return a value

```
function add(num1, num2){
    if(num1 === undefined || num2 === undefined){
        return "Please provide two numbers to add them!!!";
    }
    return num1 + num2;
}
```

```
x = 45;
y = 67;
sum = add(x, y);
document.write("<h3> sum of " + x + " and " + y + " is " + sum + ”</h3>");
```

sum of 45 and 67 is 112

Function as Variable

```
function add(num1, num2){
    if(num1 === undefined || num2 === undefined){
        return "Please provide two numbers to add them!!!";
    }
    return num1 + num2;
}
```

```
x = 45;
y = 67;
sum = add;
document.write("<h3> sum of " + x + " and " + y + " is " + sum(x, y) + ”</h3>");
```

sum of 45 and 67 is 112

Function as Variable

```
multiplication = function (num1, num2){
        if(num1 === undefined || num2 === undefined){
                return "Please provide two numbers to add them!!!";
        }
        return num1 * num2;
}
```

```
x = 45;
y = 67;
document.write("<h3> Multiplication of " + x + " and " + y + " is " + multiplication(x, y) + ”</h3>");
```

Multiplication of 45 and 67 is 3015

```
multiply(3, 4);
```

```
▶ Uncaught ReferenceError: multiply is not defined
    at first.js:136:1
```

Function

- Divide and conquer
- Code reuse
- Increase Code manageability
- Modularity
- Abstraction
- Functions are Objects

```
console.log(typeof multiplication);
```

`typeof` returns '**function**' for functions

Function

- Lamda Function or arrow function
- Self invoking functions

More details

Reference

- https://www.w3schools.com/js/default.asp
- https://playcode.io/javascript/symbols